

05_SRP

July 11, 2026

1 Sparse Random Projection (SRP)

This notebook applies **Sparse Random Projection (SRP)** to the processed TCGA PANCAN gene-expression dataset. The objective is to evaluate SRP as a data-independent, sparse linear dimension reduction method for multiclass cancer classification.

Model performance is evaluated using **Repeated Stratified 5-Fold Cross-Validation** with **5 repetitions** (25 train/test splits). To prevent data leakage, feature standardization and the SRP projection are fitted exclusively on the training data within each cross-validation split.

In addition to predictive performance, this notebook evaluates the computational efficiency, representation quality (pairwise distance preservation), and reconstruction ability of SRP.

1.1 Workflow

1. Load the processed dataset.
2. Import the required libraries.
3. Define the cross-validation strategy:
 - Repeated Stratified 5-Fold Cross-Validation (5 folds $\times$ 5 repetitions).
4. Define the numbers of projected dimensions:
 - `n_components = [5, 10, 25, 50]`
5. For each value of `n_components`:
 - For each cross-validation split:
 - Split the data into training and test sets.
 - Fit `StandardScaler` on the training data.
 - Transform the training and test data.
 - Fit `SparseRandomProjection` on the standardized training data.
 - Transform the training and test data.
 - Train the Logistic Regression classifier.
 - Generate predictions.
 - Compute and store: Accuracy, Macro Precision, Macro Recall, Macro F1 Score, Macro ROC-AUC.
 - Record: SRP fit time, SRP transform time.
 - Reconstruct the standardized test data using the Moore-Penrose pseudo-inverse of the (dense) projection matrix.
 - Compute and store the reconstruction error (MSE).
6. Aggregate the results across all cross-validation splits for each value of `n_components` by computing their mean and standard deviation.
7. Save the evaluation results:

- srp_classification_summary.csv
 - srp_reconstruction_summary.csv
 - srp_runtime_summary.csv
8. Evaluate representation quality using the full processed dataset:
 - Standardize the entire dataset.
 - Fit SRP for each value of n_components.
 - Compute pairwise distances in the original and projected feature spaces.
 - Compute the Pearson correlation between original and projected pairwise distances (distance preservation).
 - Generate and save the distance preservation plot.

1.2 1. Data and Imports

```
[1]: # Import required libraries
import warnings
warnings.filterwarnings("ignore")

import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.random_projection import SparseRandomProjection
from sklearn.metrics import pairwise_distances

random_state = 42

[2]: # Set project root
import sys
from pathlib import Path

project_root = Path("..").resolve()
sys.path.insert(0, str(project_root))

[3]: # Import custom evaluation functions
from src.evaluation import evaluate_classification, summarize_results

[4]: # Load the processed dataset
X = pd.read_csv(project_root / "data" / "processed" / "X_processed.csv")
X = X.iloc[:, 1:]
y = pd.read_csv(project_root / "data" / "processed" / "y_processed.
↳ csv", index_col=0).iloc[:, 0]
```

1.3 2. Define Experimental Settings

```
[5]: # Cross-validation strategy
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=random_state)
```

```
[6]: # Numbers of principal components to evaluate
n_components_list = [5, 10, 25, 50]
```

```
[7]: # Classifier
classifier = LogisticRegression(max_iter=1000, random_state=random_state)
```

1.4 3. SRP Classification Pipeline

```
[8]: # Start total timer
total_start = time.perf_counter()

# Initialize result storage

# Classification Performance
classification_results = {}
classification_summary = {}

# Reconstruction
reconstruction_results = {}
reconstruction_summary = {}

# Runtime
runtime_results = {}
runtime_summary = None

# Evaluate SRP using repeated stratified cross-validation
for n_components in n_components_list:

    print(f"\nEvaluating srp with {n_components} components")

    # Start timer for this value of n_components
    component_start = time.perf_counter()

    # Store results for all CV splits
    classification_split_results = []
    reconstruction_split_results = []

    fit_times = []
    transform_times = []
    srp_process_times = []

    # Cross-validation loop
```

```

for split_idx, (train_idx, test_idx) in enumerate(cv.split(X, y), start=1):

    # Split data
    X_train = X.iloc[train_idx]
    X_test = X.iloc[test_idx]

    y_train = y.iloc[train_idx]
    y_test = y.iloc[test_idx]

    # Standardisation
    scaler = StandardScaler()

    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # SRP
    srp = ↳ SparseRandomProjection(n_components=n_components, random_state=random_state)

    # Fit time
    fit_start = time.perf_counter()

    X_train_srp = srp.fit_transform(X_train_scaled)

    fit_time = time.perf_counter() - fit_start
    fit_times.append(fit_time)

    # Transform time
    transform_start = time.perf_counter()

    X_test_srp = srp.transform(X_test_scaled)

    transform_time = time.perf_counter() - transform_start
    transform_times.append(transform_time)

    # SRP process time (fit + transform)
    srp_process_time = fit_time + transform_time
    srp_process_times.append(srp_process_time)

    # Classification

    classifier.fit(X_train_srp, y_train)

    y_pred = classifier.predict(X_test_srp)
    y_prob = classifier.predict_proba(X_test_srp)

    metrics = evaluate_classification(

```

```

        y_test,
        y_pred,
        y_prob,
    )

    metrics["Split"] = split_idx
    classification_split_results.append(metrics)

# Reconstruction

    components = srp.components_.toarray()
    projection_pinv = np.linalg.pinv(components.T)

    X_test_reconstructed = X_test_srp @ projection_pinv
    X_test_reconstructed = scaler.inverse_transform(X_test_reconstructed)

    reconstruction_error = np.mean(
        (X_test.to_numpy() - X_test_reconstructed) ** 2
    )

    reconstruction_split_results.append(
        {
            "Split": split_idx,
            "Reconstruction MSE": reconstruction_error,
        }
    )

# Store classification results
    classification_results[n_components] = pd.
    ↪DataFrame(classification_split_results)

    classification_summary[n_components] = summarize_results([
        {k: v for k, v in m.items() if k != "Split"} for m in classification_split_results])

# Store reconstruction results
    reconstruction_results[n_components] = pd.
    ↪DataFrame(reconstruction_split_results)

    reconstruction_summary[n_components] = (
        reconstruction_results[n_components]
        .drop(columns="Split")
        .agg(["mean", "std"])
        .T
        .rename(columns={"mean": "Mean", "std": "Std"})
    )

```

```

# Runtime
component_end = time.perf_counter()

runtime_results[n_components] = {
    "Fit Time Mean (s)": np.mean(fit_times),
    "Fit Time Std (s)": np.std(fit_times, ddof=1),
    "Transform Time Mean (s)": np.mean(transform_times),
    "Transform Time Std (s)": np.std(transform_times, ddof=1),
    "SRP Process Time Mean (s)": np.mean(srp_process_times),
    "SRP Process Time Std (s)": np.std(srp_process_times, ddof=1),
    "Total Runtime (s)": component_end - component_start,
}

print(
    f"srp ({n_components}>2} components): "
    f"{runtime_results[n_components]['Total Runtime (s)']:.2f} seconds"
)

# Runtime summary
runtime_summary = pd.DataFrame(runtime_results).T
runtime_summary.index.name = "n_components"

# Total runtime
total_end = time.perf_counter()

print(f"\nTotal runtime: {(total_end - total_start)/60:.2f} minutes")

```

Evaluating srp with 5 components
 srp (5 components): 84.31 seconds

Evaluating srp with 10 components
 srp (10 components): 88.12 seconds

Evaluating srp with 25 components
 srp (25 components): 106.25 seconds

Evaluating srp with 50 components
 srp (50 components): 92.51 seconds

Total runtime: 6.19 minutes

1.5 4. Results Summary

```
[9]: # Combine classification summaries
classification_summary_df = pd.concat(
    classification_summary,
    names=["n_components", "Metric"]
)

print(classification_summary_df)
```

		Mean	Std
n_components	Metric		
5	Accuracy	0.581005	0.033690
	Macro Precision	0.590983	0.041856
	Macro Recall	0.562397	0.033343
	Macro F1	0.567652	0.034974
	Macro ROC AUC	0.851697	0.018461
10	Accuracy	0.783758	0.024296
	Macro Precision	0.759740	0.033928
	Macro Recall	0.757211	0.028593
	Macro F1	0.755250	0.029955
	Macro ROC AUC	0.948324	0.009635
25	Accuracy	0.885148	0.015157
	Macro Precision	0.884752	0.015509
	Macro Recall	0.893893	0.014554
	Macro F1	0.887211	0.014279
	Macro ROC AUC	0.976137	0.006347
50	Accuracy	0.942823	0.019804
	Macro Precision	0.937713	0.024173
	Macro Recall	0.946831	0.018280
	Macro F1	0.941037	0.021475
	Macro ROC AUC	0.994116	0.003442

```
[10]: # Combine reconstruction summaries

reconstruction_summary_df = pd.concat(
    reconstruction_summary,
    names=["n_components", "Metric"]
)

print(reconstruction_summary_df)
```

		Mean	Std
n_components	Metric		
5	Reconstruction MSE	1.787953	0.025394
10	Reconstruction MSE	1.787160	0.025367
25	Reconstruction MSE	1.786252	0.025347
50	Reconstruction MSE	1.784354	0.025366

```
[11]: print(runtime_summary)
```

	Fit Time Mean (s)	Fit Time Std (s)	Transform Time Mean (s)	\
n_components				
5	0.047886	0.005400	0.005752	
10	0.055698	0.007055	0.006008	
25	0.071158	0.011000	0.006680	
50	0.102967	0.014786	0.007985	

	Transform Time Std (s)	SRP Process Time Mean (s)	\
n_components			
5	0.000758	0.053638	
10	0.000716	0.061705	
25	0.001898	0.077838	
50	0.001906	0.110953	

	SRP Process Time Std (s)	Total Runtime (s)
n_components		
5	0.005858	84.311880
10	0.007451	88.119596
25	0.012239	106.245064
50	0.015965	92.510952

```
[12]: # Save Results
# Output directories

metrics_dir = Path("../results/metrics")
timings_dir = Path("../results/timings")

metrics_dir.mkdir(parents=True, exist_ok=True)
timings_dir.mkdir(parents=True, exist_ok=True)

# Save summaries

classification_summary_df.to_csv(
    metrics_dir / "srp_classification_summary.csv"
)

reconstruction_summary_df.to_csv(
    metrics_dir / "srp_reconstruction_summary.csv"
)

runtime_summary.to_csv(
    timings_dir / "srp_runtime_summary.csv"
)

print("Results saved successfully.")
```


Results saved successfully.

2 5. Representation Quality

```
[13]: # Standardize the full dataset
scaler_full = StandardScaler()
X_scaled = scaler_full.fit_transform(X)

[14]: # Figure Directories
figure_dir = Path("../figures/SRP")
figure_dir.mkdir(parents=True, exist_ok=True)

[15]: # Distance Preservation
# Representation quality results
distance_results = []

# Evaluate distance preservation for each number of components
for n_components in n_components_list:

    srp = SparseRandomProjection(
        n_components=n_components,
        random_state=random_state,
    )

    X_srp = srp.fit_transform(X_scaled)

    # Compute pairwise distances
    original_distances = pairwise_distances(X_scaled)
    projected_distances = pairwise_distances(X_srp)

    # Correlation between original and projected distances
    distance_correlation = np.corrcoef(
        original_distances.ravel(),
        projected_distances.ravel(),
    )[0, 1]

    distance_results.append({
        "n_components": n_components,
        "Distance Correlation": distance_correlation,
    })

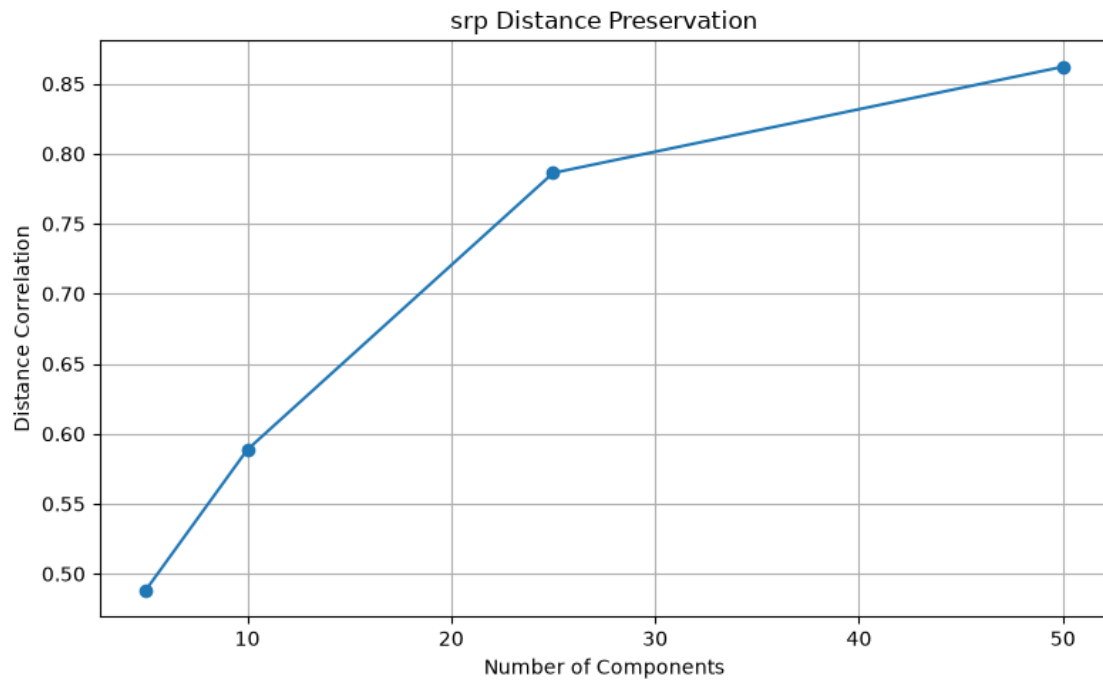
# Save results
distance_df = pd.DataFrame(distance_results)

distance_df.to_csv(
    metrics_dir / "srp_distance_preservation.csv",
    index=False,
```

```
)  
  
print(distance_df)
```

	n_components	Distance Correlation
0	5	0.487961
1	10	0.588652
2	25	0.786298
3	50	0.862146

```
[16]: # Distance Preservation Plot  
plt.figure(figsize=(8,5))  
  
plt.plot(  
    distance_df["n_components"],  
    distance_df["Distance Correlation"],  
    marker="o",  
)  
  
plt.xlabel("Number of Components")  
plt.ylabel("Distance Correlation")  
plt.title("srp Distance Preservation")  
  
plt.grid(True)  
  
plt.tight_layout()  
  
plt.savefig(  
    figure_dir / "distance_preservation.png",  
    dpi=300,  
    bbox_inches="tight",  
)  
  
plt.show()
```



[]: